

Hunting In Memory (Lab 2)

LAB 7.2

Scenario

The organization you work for has also asked you to perform memory threat hunting on a compromised Linux machine. As a hunting exercise to keep you sharp, the IT Security manager tasked you specifically with looking for the existence of Linux rootkits

Goals

The learning objective of this lab is to get familiar with advanced usage of Volatility.

What you will learn

You will learn how to utilize a memory dump to confirm a compromise by a rootkit.

Recommended tools

- Volatility

Tasks

Task 1. Is the system infected by Diamorphine?

[Diamorphine](#) (developed by Victor Mello) is a kernel rootkit written in C that supports Linux Kernels 2.6.x/3.x/4.x. It can hide processes, files and directories. It works by hooking the *sys_call_table*, more specifically it hooks the *kill*, *getdents* and *getdents64* syscall handler addresses, making them point to the Diaphormine code. After loading into memory, the Loadable Kernel Module won't be visible in */proc/modules* but is still visible under */sys/module*.

For this task you will work against the *vanilla.memory* and *infection1.memory* memory images residing on */root/memory_dump*. *vanilla.memory* is a memory image of the system in pristine condition.

In order to check if the system was infected by Diamorphine, leverage Volatility's *linux_check_modules* and *linux_check_syscall* plugins as well as *VolShell*.

Task 2. Is the system infected by Reptile?

[Reptile](#) (developed by Ighor Augusto) is a feature rich rootkit. It is written in C and under the hood it uses the Khook framework. Khook, among other things, instead of hooking the *sys_call_table* it uses a different technique that patches a function prologue with a JMP instruction

For this task you will work against the *vanilla.memory* and *infection2.memory* memory images residing on */root/memory_dump*. *vanilla.memory* is a memory image of the system in pristine condition.

In order to check if the system was infected by Diamorphine, leverage Volatility's *linux_hidden_modules* and *linux_check_inline_kernel* plugins as well as *VolShell*.

Volatility can be found inside the */root/memory_dump* directory

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Is the system infected by Diamorhine?

The *linux_check_modules* plugin looks for Loadable Kernel Modules (LKM) that are not listed under */proc/module* but still appear under */sysfs/module* and will output such discrepancies.

The *linux_check_syscall* plugin checks for modifications of the *sys_call_table*. All syscall handler function pointers are listed in the *sys_call_table* array. This plugin compares them with the address specified in the Kernel Symbols Table. In case of a mismatch, the message "hooked" is displayed.

First let's execute the below (inside */root/memory_dump*) to identify all available profiles.

```
vol.py --info
```

We'll use the one depicted below.

```
root@attackdefense:~# vol.py --info
Volatility Foundation Volatility Framework 2.6.1

Profiles
-----
Linuxprofile-2_6_32-754_el6_x86_64x64 - A Profile for Linux profile-2.6.32-754.el6.x86_64 x64
```

Now, let's start by executing the *linux_check_modules* plugin against *infection1.memory*.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_check_modules -f /root/memory_dump/infection1.memory
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_check_modules
/root/memory_dump/infection1.memory
Volatility Foundation Volatility Framework 2.6.1
  Module Address      Core Address      Init Address  Module Name
-----
0xfffffffffa0523740 0xfffffffffa0523000      0x0  diamorphine
root@attackdefense:~#
```

Diamorphine's module name was identified (it was still visible under */sysfs/module*). Let's use *VolShell* to look at the first bytes of that module.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection1.memory
Then, we use the db command as follows.
>>> db(0xfffffffffa0523740,128)
```

0xfffffffffa0523740 is the module address we found with the help of *linux_check_modules*.

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root
/memory_dump/infection1.memory
Volatility Foundation Volatility Framework 2.6.1
Current context: process init, pid=1 DTB=0x137638000
Welcome to volshell! Current memory image is:
file:///root/memory_dump/infection1.memory
To get help, type 'hh()'
>>> db(0xfffffffffa0523740,128)
0xfffffffffa0523740  00 00 00 00 00 00 00 00 00 00 01 10 00 00 00 ad de  .....
0xfffffffffa0523750  00 02 20 00 00 00 ad de 64 69 61 6d 6f 72 70 68  .....diamorph
0xfffffffffa0523760  69 6e 65 00 00 00 00 00 00 00 00 00 00 00 00 00  ine.....
0xfffffffffa0523770  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0xfffffffffa0523780  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0xfffffffffa0523790  e0 1f 3d a6 00 88 ff ff b8 7f 52 a0 ff ff ff ff  ..=.....R.....
0xfffffffffa05237a0  18 6f 51 a0 ff ff ff ff d8 9c 80 3d 01 88 ff ff  .oQ.....=.....
0xfffffffffa05237b0  c0 9c 80 3d 01 88 ff ff c0 74 ab 81 ff ff ff ff  ...=.....t.....
>>> 
```

Let's now use *linux_check_syscall*, as follows.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_check_syscall -f /root/memory_dump/infection1.memory --output-file=linux_check_syscall.txt
cat linux_check_syscall.txt | grep -i hooked
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_check_syscall -f /root
/memory_dump/infection1.memory --output-file=linux_check_syscall.txt
Volatility Foundation Volatility Framework 2.6.1
Outputting to: linux_check_syscall.txt
root@attackdefense:~# cat linux_check_syscall.txt | grep -i hooked
64bit          62          0xffffffffffa0523190 HOOKED: diamorphine/hacked_kill
64bit          78          0xffffffffffa0523230 HOOKED: diamorphine/hacked_getdents
64bit          217         0xffffffffffa0523420 HOOKED: diamorphine/hacked_getdents64
root@attackdefense:~# █
```

linux_check_syscall, identified three hooked syscalls (62, 78 and 217). The machine we are currently at has been disinfected, so we can check the syscall table for the abovementioned syscalls (*sys_kill*, *sys_getdents* and *sys_getdents64*), as follows.

```
cat /proc/kallsyms | grep 'sys_getdents\|sys_kill'
```

```
root@attackdefense:~# cat /proc/kallsyms | grep 'sys_getdents\|sys_kill'
0000000000000000 T __x64_sys_kill
0000000000000000 T __ia32_sys_kill
0000000000000000 T __ia32_compat_sys_getdents
0000000000000000 T __ia32_sys_getdents
0000000000000000 T __x32_compat_sys_getdents
0000000000000000 T __x64_sys_getdents
0000000000000000 T ksys_getdents64
0000000000000000 T __x64_sys_getdents64
0000000000000000 T __ia32_sys_getdents64
0000000000000000 t _eil_addr__ia32_sys_kill
0000000000000000 t _eil_addr__x64_sys_kill
0000000000000000 t _eil_addr__x32_compat_sys_getdents
0000000000000000 t _eil_addr__ia32_compat_sys_getdents
0000000000000000 t _eil_addr__ia32_sys_getdents64
0000000000000000 t _eil_addr__x64_sys_getdents64
0000000000000000 t _eil_addr__ia32_sys_getdents
0000000000000000 t _eil_addr__x64_sys_getdents
root@attackdefense:~#
```

Let's look at how *sys_kill* looks like in the pristine system, as follows.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/vanilla.memory
```

Then, we use the *dis* command as follows.

```
dis(0xffffffff81098d20,length=45)
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/vanilla.memory
Volatility Foundation Volatility Framework 2.6.1
Current context: process init, pid=1 DTB=0x137638000
Welcome to volshell! Current memory image is:
file:///root/memory_dump/vanilla.memory
To get help, type 'hh()'
>>> dis(0xffffffff81098d20,length=45)
0xffffffff81098d20 55          PUSH RBP
0xffffffff81098d21 4889e5        MOV RBP, RSP
0xffffffff81098d24 4881ecb0000000 SUB RSP, 0xb0
0xffffffff81098d2b 48895dd8      MOV [RBP-0x28], RBX
0xffffffff81098d2f 4c8965e0      MOV [RBP-0x20], R12
0xffffffff81098d33 4c896de8      MOV [RBP-0x18], R13
0xffffffff81098d37 4c8975f0      MOV [RBP-0x10], R14
0xffffffff81098d3b 4c897df8      MOV [RBP-0x8], R15
0xffffffff81098d3f 0f1f440000    NOP DWORD [RAX+RAX+0x0]
0xffffffff81098d44 654c8b3c2500fc0000 MOV R15, [GS:0xfc00]
>>> □
```

This is how *sys_kill* looks like in the pristine system.

Let's now look at how *sys_kill* looks like in the infected system, as follows.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection1.memory
```

Then, we use the *dis* command as follows.

```
dis(0xfffffffffa0523190,length=45)
```

0xfffffffffa0523190 is the first hooked syscall (sys_kill) that *linux_check_syscall* identified.

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection1.memory
```

```
Volatility Foundation Volatility Framework 2.6.1
```

```
Current context: process init, pid=1 DTB=0x137638000
```

```
Welcome to volshell! Current memory image is:
```

```
file:///root/memory_dump/infection1.memory
```

```
To get help, type 'hh()'
```

```
>>>
```

```
>>> dis(0xfffffffffa0523190,length=45)
```

0xfffffffffa0523190	55	PUSH RBP
0xfffffffffa0523191	4889e5	MOV RBP, RSP
0xfffffffffa0523194	0f1f440000	NOP DWORD [RAX+RAX+0x0]
0xfffffffffa0523199	83fe3f	CMP ESI, 0x3f
0xfffffffffa052319c	7462	JZ 0xfffffffffa0523200
0xfffffffffa052319e	83fe40	CMP ESI, 0x40
0xfffffffffa05231a1	744d	JZ 0xfffffffffa05231f0
0xfffffffffa05231a3	83fe1f	CMP ESI, 0x1f
0xfffffffffa05231a6	48c7c02070a981	MOV RAX, 0x81a97020
0xfffffffffa05231ad	7411	JZ 0xfffffffffa05231c0
0xfffffffffa05231af	488b05d2070000	MOV RAX, [RIP+0x7d2]
0xfffffffffa05231b6	e895a6d8e0	CALL 0xffffffff812ad850
0xfffffffffa05231bb	c9	LEAVE
0xfffffffffa05231bc	c3	RET

```
>>> □
```

This is how the hooked *sys_kill* syscall looks like (in the infected system).

By comparing how the two syscalls (*sys_kill* in the pristine system and *sys_kill* in the infected system) you can see how a hook looks like!

Task 2. Is the system infected by Reptile?

This time we will use *the linux_hidden_modules* plugin. The *linux_check_syscal* plugin can't detect the hooking technique being employed by the Reptile rootkit (since the syscall handler addresses have not been modified).

Among other things, Reptile hooks *fillonedir()*, *filldir()*, *filldir64()*, *compat_fillonedir()*, *compat_filldir()*, *compat_filldir64()*, *__d_lookup()*. To hide processes, it hooks *tgid_iter()* and *next_tgid()*. To hide network connections, it hooks *tcp4_seq_show* and *udp4_seq_show*.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_e16_x86_64x64 linux_hidden_modules -f /root/memory_dump/infection2.memory
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_e16_x86_64x64 linux_hidden_modules -f /root/memory_dump/infection2.memory
Volatility Foundation Volatility Framework 2.6.1
Offset (V)      Name
-----
0xffffffffffa0596720 reptile
root@attackdefense:~#
```

The Reptile rootkit hidden module was uncovered by the *linux_hidden_modules* plugin.

Now, let's also check the *linux_check_inline_kernel* plugin. This plugin detects inline hooking. Among other things it checks if the prologue of specific functions in the kernel contains assembly instructions like JMP, CALL or RET and warns the analyst of any functions being hooked.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_e16_x86_64x64 linux_check_inline_kernel -f /root/memory_dump/infection2.memory
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_e16_x86_64x64 linux_check_inline_kernel -f /root/memory_dump/infection2.memory
Volatility Foundation Volatility Framework 2.6.1
Name                                     Member                                Hook Type Hook Address
-----
tcp4_seq_afinfo                         show                                JMP      0x0000000000000000
udplite4_seq_afinfo                     show                                JMP      0x0000000000000000
udp4_seq_afinfo                         show                                JMP      0x0000000000000000
TCP                                     ioctl                               JMP      0x0000000000000000
UDP                                     ioctl                               JMP      0x0000000000000000
UDP-Lite                               ioctl                               JMP      0x0000000000000000
PING                                     ioctl                               JMP      0x0000000000000000
RAW                                     ioctl                               JMP      0x0000000000000000
root@attackdefense:~#
```

The plugin detected several network-related functions that were patched by the Reptile rootkit.

Let's use VolShell again against both *vanilla.memory* and *infection2.memory* to see how a hooked function(*tcp4_seq_show*) looks like.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_e16_x86_64x64 linux_volshell -f /root/memory_dump/vanilla.memory
```

Then, we use the *dis* command as follows.

```
dis(addrspace().profile.get_symbol("tcp4_seq_show"),length=11)
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/vanilla.memory
Volatility Foundation Volatility Framework 2.6.1
Current context: process init, pid=1 DTB=0x137638000
Welcome to volshell! Current memory image is:
file:///root/memory_dump/vanilla.memory
To get help, type 'hh()'
>>> dis(addrspace().profile.get_symbol("tcp4_seq_show"),length=11)
0xffffffff814e0a80 55                                PUSH RBP
0xffffffff814e0a81 4889e5                                MOV RBP, RSP
0xffffffff814e0a84 4881ec10010000 SUB RSP, 0x110
>>> □
```

This is how *tcp4_seq_show* looks like in the pristine system.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection2.memory
```

Then, we use the *dis* command as follows.

```
dis(addrspace().profile.get_symbol("tcp4_seq_show"),length=11)
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection2.memory
Volatility Foundation Volatility Framework 2.6.1
Current context: process init, pid=1 DTB=0x136c6a000
Welcome to volshell! Current memory image is:
file:///root/memory_dump/infection2.memory
To get help, type 'hh()'
>>> dis(addrspace().profile.get_symbol("tcp4_seq_show"),length=11)
0xffffffff814e0a80 e92bbd0b1f          JMP 0xfffffffffa059c7b0
0xffffffff814e0a85 81ec10010000        SUB ESP, 0x110
>>> █
```

This is how *tcp4_seq_show* looks like in the infected system. It has been patched to jump (JMP) to the Reptile code.

Note: In order to hide directories Reptile also patches the *fillonedir* function. Volatility didn't detect this!

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/vanilla.memory
Then, we use the dis command as follows.
dis(addrspace().profile.get_symbol("fillonedir"),length=11)
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/vanilla.memory
Volatility Foundation Volatility Framework 2.6.1
Current context: process init, pid=1 DTB=0x137638000
Welcome to volshell! Current memory image is:
file:///root/memory_dump/vanilla.memory
To get help, type 'hh()'
>>> dis(addrspace().profile.get_symbol("fillonedir"),length=11)
0xffffffff811b46d0 55          PUSH RBP
0xffffffff811b46d1 4889e5      MOV RBP, RSP
0xffffffff811b46d4 4883ec30    SUB RSP, 0x30
0xffffffff811b46d8 48          DB 0x48
0xffffffff811b46d9 89          DB 0x89
0xffffffff811b46da 5d          POP RBP
>>> █
```

This is how *fillonedir* looks like in the pristine system.

```
vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection2.memory
```

Then, we use the *dis* command as follows.

```
dis(addrspace().profile.get_symbol("fillonedir"),length=11)
```

```
root@attackdefense:~# vol.py --plugins=plugins --profile=Linuxprofile-2_6_32-754_el6_x86_64x64 linux_volshell -f /root/memory_dump/infection2.memory
Volatility Foundation Volatility Framework 2.6.1
Current context: process init, pid=1 DTB=0x136c6a000
Welcome to volshell! Current memory image is:
file:///root/memory_dump/infection2.memory
To get help, type 'hh()'
>>> dis(addrspace().profile.get_symbol("fillonedir"),length=11)
0xffffffff811b46d0 e95b793e1f          JMP 0xfffffffffa059c030
0xffffffff811b46d5 83ec30          SUB ESP, 0x30
0xffffffff811b46d8 48             DB 0x48
0xffffffff811b46d9 89             DB 0x89
0xffffffff811b46da 5d            POP RBP
>>> █
```

This is how *fillonedir* looks like in the infected system. It has been patched to jump (JMP) to the Reptile code.

References:

1. <http://www.dfir.org/research/android-memory-analysis-DI.pdf>
2. <https://www.youtube.com/watch?v=oWkOyphlmM8>
3. <https://github.com/504ensicsLabs/LiME>
4. <https://www.blackhat.com/presentations/bh-dc-07/Walters/Paper/bh-dc-07-Walters-WP.pdf>
5. http://4tphi.net/fatkit/papers/fatkit_journal.pdf
6. <http://volatilesystems.blogspot.com/2008/08/pyflagvolatility-team-wins-dfrws.html>
7. <http://dfir.org/research/omfw.pdf>
8. <https://github.com/halpomeranz/lmg>
9. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>
10. <https://github.com/volatilityfoundation/volatility/wiki/Linux-Command-Reference>

11. <https://github.com/m0nad/Diamorphine>
12. <https://github.com/f0rb1dd3n/Reptile/>
13. <https://github.com/h2hconference/2018/>
14. <https://countuponsecurity.com/2019/10/14/notes-on-linux-memory-analysis-lime-volatility-and-lkms/>